University of Twente

Nth time's the charm

NWERC 2026 team reference

Index Page numbers; letter codes identify formula entries.

Python reference (2–4)

Arithmetic, Python	**2**	Functions / scope, Python	**2**
Arrays / lists	**2**	Heaps / merging	**4**
Big integers	**2**	Input / output	**2**
Bitmasks	**3**	Input formats	**2**
Caching	**4**	JSON	**4**
Calendar / dates	**4**	Queues / counters	**3**
Combinatorial iterators	**3**	Sorting / binary search	**3**
Complexities, Python	**4**	Strings / regex	**3**
Decimals / fractions	**2–3**	Topological order	**4**
Dictionaries / sets	**3**		

C++ reference (5–6)

C++ containers	**5**
C++ input / output	**5**
C++ numeric pitfalls	**6**
C++ sorting / searching	**6**
C++ syntax / references	**5**

Algorithms and mathematics (7–11)

Bell numbers	**9 C1**	Exponent reduction	**8 N3,5**	Pair distances / sums	**9 S2**
Binomial identities	**9 B1**	Factorials / zeros	**9 B2**	Permutation cycles	**9 C2**
Binomial, prime powers	**9 B4**	Fibonacci	**9 S3**	Planar graphs	**8 G7**
Binomial probability	**10 P3**	Floor quotients	**8 D3**	Probability tails	**10 P2**
Bracelets	**9 C4**	Four-cycles	**8 G3**	Prüfer / labeled trees	**8 G5**
Burnside / orbits	**9 C3**	Games / nim	**11**	Recurrences	**9 S3–4**
Congruences / CRT	**8 N4**	Gcd pair sums	**8 D2**	Searching / greedy	**10**
Coprime pairs	**8 D2**	Generating functions	**9 S4**	Spanning trees	**8 G6**
Counting / Catalan	**7**	Geometric distribution	**10 P4**	Stirling numbers	**9 C1**
Covariance	**10 P1**	Geometry formulas	**7**	String matching	**11**
Cycles in graphs	**8 G1–4**	Graph tools	**10**	Testing / stress	**11**
Diophantine equations	**7 N1**	Hypergeometric	**10 P5**	Totient, Euler φ	**8 N2**
Divisor sums	**8 D1–3**	Inverses, modular	**8 N3**	Trees / ancestors	**11**
Dynamic programming	**10**	Lucas theorem	**9 B3**	Triangles	**8 G2**
Elementary sums	**7**	Markov / Chebyshev	**10 P6**	Variance	**10 P1**
Exact modular division	**9 B5**	Möbius / inversion	**8 D1**	Walks, adjacency	**8 G3**
Expectation	**10 P1**	Necklaces	**9 C4**	Weighted sums	**9 S1**

Notation: $\sum$ sum; $\prod$ product; $a \mid b$: a divides b; $\gcd$: greatest common divisor; lcm: least common multiple; $\lfloor x \rfloor$: round down; $\lceil x \rceil$: round up; $\binom{n}{k}$: choose k of n; $[P]$: 1 if P holds, else 0.

1 Workspace and setup

1.1 Directory structure

```
contest/
    tmpl/
        1.in
        1.out
        py.py
        cpp.cpp
    s.sh
```

1.2 Create and run

```
mkdir -p contest/tmpl
cd contest              # save files as shown
source s.sh             # repeat in each terminal
tmpl                    # copy tmpl/ into A-N
rp A                    # Python, terminal input
rc A                    # compile and run C++
rp A 1                  # reads A/1.in
rc A 1
tp A                    # test Python on all A/*.in
tc A                    # compile once, test all C++
```

Pair `A/1.in` with expected `A/1.out`. Replace the copied samples for each problem. All cases run; any failure gives a nonzero status. Spacing/blank lines are ignored; custom or floating-point outputs need manual checks.

`tmpl` keeps existing files. Letters accept either case. `rp A sample` reads `A/sample.in`; no input argument uses stdin.

1.3 s.sh (Bash)

```
tmpl() {
    local p
    for p in {A..N}; do
        cp -rn tmpl/. "$p"
    done
}
_c() { g++ -std=c++17 -O2 "$1/cpp.cpp" -o "$1/$1"; }
rc() {
    local p=${1^^} input=/dev/stdin
    [ "$#" -gt 1 ] && input="$p/$2.in"
    _c "$p" && "./$p/$p" < "$input"
}
rp() {
    local p=${1^^} input=/dev/stdin
    [ "$#" -gt 1 ] && input="$p/$2.in"
    python3 "$p/py.py" < "$input"
}
_t() (
    set -o pipefail
    shopt -s failglob
    bad=0
    for f in "$1"/*.in; do
        echo "$f"
        "${@:2}" < "$f" | diff -bB "${f%.in}.out" - &&
            echo OK || { echo FAIL; bad=1; }
    done
    exit "$bad"
)
tp() { _t "${1^^}" rp "$1"; }
tc() {
    local p=${1^^}
    _c "$p" && _t "$p" "./$p/$p"
}
```

1.4 tmpl/cpp.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define dbg(x) cout << (x) << '\n'

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
}
```

1.5 tmpl/py.py

```
dbg = print
def ints(): return list(map(int, input().split()))
```

`ints()` returns a list: `n,m=ints()` or `a=ints()`. Remove `dbg(...)` calls before testing or submitting.

1.6 Python imports

Import as needed: `from module import name`

Module	Useful names
`collections`	`Counter, defaultdict, deque`
`operator`	`itemgetter, attrgetter`
`heapq`	`heappush, heappop, heapify`
`bisect`	`bisect_left, bisect_right`
`functools`	`cache, cmp_to_key`
`itertools`	`accumulate, combinations, permutations, product`
`math`	`gcd, lcm, isqrt, comb, inf, sqrt, hypot, atan2, pi, factorial, prod`

2 Python: essentials

Python 3.10+; standard library only. Run: `python3 sol.py < 1.in`.

2.1 Input and output

```
n = int(input())
a = list(map(int, input().split()))
print(sum(a))
print(*a)                           # space-separated
```

Built-in `input()` returns a string without its final newline. Keep it as the default. Optional faster line input still returns strings:

```
# import sys
# input = sys.stdin.readline  # optional; keeps newline
# write = sys.stdout.write    # optional; adds no newline
```

`readline()` returns `""` at EOF; built-in `input()` raises `EOFError`. Strip line endings for text comparisons with `input().rstrip("\r\n")`. Add these aliases only when needed; the template uses built-in `input()`.

Optional whole-file integer input (not interactive; uses more memory):

```
import sys
it = iter(map(int, sys.stdin.buffer.read().split()))
n = next(it)
a = [next(it) for _ in range(n)]
```

`buffer` gives bytes; use `token.decode()` for text. Interactive: `print("?", x, flush=True)`.

2.2 Control flow and functions

```
def bounds(a):
    if not a:
        return None
    return min(a), max(a)        # tuple return

for i, x in enumerate([3, 1, 4]):
    if x < 0:
        continue
    if x == 4:
        break
else:
    print("no break")

x = 20
while x > 0:
    x //= 2
```

Function assignment creates a local variable. Use `nonlocal` for a name in an enclosing function, `global` for a module name. Mutating a passed list changes that list; rebinding the parameter does not. Use `def f(a=None)` and create the list inside; `a=[]` as a function default reuses the same list across calls.

2.3 Input formats and output batches

```
t = int(input())                # only if t is specified
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))

# Alternative: read lines until EOF (no test count).
import sys
for line in sys.stdin:
    if not line.strip():
        continue
    a = list(map(int, line.split()))

out = [str(x*x) for x in range(5)]
print("\n".join(out))
# Alternative using the optional write alias:
# write("\n".join(out) + "\n")
```

These are separate input formats: follow the statement. Do not assume one integer per line. For a fixed number of tokens spread across lines, use the optional whole-file reader above. Interactive: read one reply per question and flush each question.

2.4 Arithmetic and comparisons

```
7 / 3                  # float: 2.333...
-7 // 3, -7 % 3        # (-3, 2): division floors
a, b = -7, 3           # b > 0
ceil_div = -(-a // b)  # -2, works for either sign of a
2 ** 10                # 1024; ^ is XOR, not power
10 ** 18               # exact int; 1e18 is a float
x, y = 3, 8; x, y = y, x  # swap; unpack a tuple
0 <= x < 10            # chained comparison
x == y                 # value equality; "is": identity
```

2.5 Lists, slices and grids

```
a = [10, 20, 30, 40]
a.append(50); a.extend([60, 70])
last = a.pop()                  # removes last
a[-1], a[1:3], a[::-1]          # 60, [20,30], reversed
b = a.copy()                    # shallow copy
a[1:3] = [9]                    # replace a slice
n, m = 3, 4
grid = [[0] * m for _ in range(n)]
squares = [x*x for x in range(6) if x % 2 == 0]
```

a=b aliases; `[[0]*m]*n` repeats the same row. Slices copy and exclude the stop index.

2.6 Loops and built-ins

```
a = [3, 1, 4]
list(range(2, 8, 2))            # [2, 4, 6]
list(enumerate(a))              # [(0,3),(1,1),(2,4)]
list(zip(a, [10, 20]))          # stops at shorter input
len(a), sum(a), min(a), max(a)  # 3, 8, 1, 4
any(x < 0 for x in a)           # False
all(x > 0 for x in a)           # True
list(reversed(a))               # [4, 1, 3]
```

`map`/`zip`/`reversed` are one-pass iterators; `list(...)` stores their results.

3 Python: exact numbers

3.1 Huge integers and modular arithmetic

```
a = 10 ** 100                   # arbitrary-size int
q, r = divmod(a, 97)            # a == q*97 + r
int("ff", 16), int("101", 2)    # 255, 5
bin(13), hex(255)               # '0b1101', '0xff'
pow(2, 1000, 97)                # 2**1000 mod 97
pow(3, -1, 10)                  # 7: inverse, gcd=1
abs(-123), len(str(a))          # 123, 101 digits
```

`pow(a,e,m)` avoids building `a**e`. Inverses need coprime `a,m`; `m` need not be prime.

For extremely large decimal input/output, CPython may limit `int`/`str` conversion (often 4300 digits). For a trusted contest problem requiring more digits, optionally disable that conversion limit:

```
import sys
if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)
```

3.2 Integer math and counting

```
from math import gcd, lcm, isqrt, comb, perm, factorial
gcd(12, 18, 30), lcm(4, 6, 10)    # 6, 60
isqrt(10 ** 100 + 1)              # 10**50, exact floor
comb(100, 50)                     # exact n choose k
perm(5, 2), factorial(5)          # 20, 120
from math import inf, pi, hypot, atan2, floor, ceil
hypot(3, 4), atan2(1, 0)          # 5.0, pi/2 radians
```

`comb`/`factorial` give full integers, not values modulo `MOD`. Use `isqrt`, not float `sqrt`, for exact roots.

3.3 Exact fractions

```
from fractions import Fraction as F
x = F(1, 3) + F(1, 6)             # Fraction(1, 2)
x.numerator, x.denominator        # (1, 2)
F("0.125")                        # 1/8, exact decimal
F(2, 3) < F(3, 4)                 # exact comparison
F("3.14159").limit_denominator(100)  # 311/99
```

Fractions reduce automatically. Use integers or strings: `F(0.1)` preserves a binary float, not exactly `1/10`.

3.4 High-precision decimal arithmetic

```
from decimal import Decimal as D, localcontext
with localcontext() as ctx:
    ctx.prec = 60                 # significant digits
    x = D(1) / D(7)               # rounded to precision
    root = D(2).sqrt()
    y = D("0.1") + D("0.2")       # exactly 0.3
    z = D("1.235").quantize(D("0.01"))  # 1.24
```

Use strings for decimal input. Context controls precision and rounding. For exact `1/3`, use `Fraction`.

3.5 Integer bitmasks

```
mask, i, n = 0b10110, 1, 6
mask.bit_count(), mask.bit_length()  # 3, 5
bool(mask & (1 << i))             # test bit i
mask |= 1 << 5                    # set bit 5
mask &= ~(1 << 1)                 # clear bit 1
mask ^= 1 << 2                    # toggle bit 2
mask << 3, mask >> 1              # unlimited-width shifts
(~mask) & ((1 << n) - 1)          # complement in n bits
```

4 Python: containers

4.1 Dictionaries and composite keys

```
d = {"a": 2}
d["b"] = 3
d.get("missing", 0)               # 0; does not insert
"a" in d                          # membership tests keys
list(d.items())                   # [('a',2),('b',3)]
list(d.keys()), list(d.values())
d.pop("missing", None)            # default avoids KeyError
positions = {(2, 3): 7}           # tuple key, built-in hash
frozen = frozenset([1, 2, 2])     # hashable set: {1,2}
```

Dict iteration preserves insertion order. Keys must be hashable: use tuples/frozensets, not lists/sets. Missing `d[k]` raises `KeyError`.

4.2 Sets

```
s, t = {1, 2, 3}, {3, 4}
s | t, s & t                      # union, intersection
s - t, s ^ t                      # difference, symmetric diff
s <= t, s.isdisjoint(t)           # subset, no overlap
s.add(5); s.discard(99)           # discard tolerates missing
empty = set()                     # {} is an empty dict
unique = set([3, 1, 3])
```

Sets have no sorted-order guarantee. Set/dict membership is average $O(1)$; list membership is $O(n)$.

4.3 Frequency tables and multisets

```
from collections import Counter
c = Counter("banana")             # a:3, n:2, b:1
c["x"]                            # 0; does not insert
c.most_common(2)                  # [('a',3), ('n',2)]
list(c.elements())                # repeat by multiplicity
c + Counter("an"), c - Counter("an")
c & Counter("band"), c | Counter("band")
# &: minimum counts; |: maximum; +/-: add/subtract
```

Counter arithmetic drops nonpositive counts. `c.subtract(other)` mutates and keeps zero/negative counts.

4.4 Grouping with missing defaults

```
from collections import defaultdict
groups = defaultdict(list)
groups["a"].append(4)             # creates [] if missing
counts = defaultdict(int)
counts["a"] += 1                  # creates 0 if missing
sets = defaultdict(set)
sets["a"].add(4)
```

4.5 Deque: queue and both ends

```
from collections import deque
q = deque([1, 2])                 # O(1) ends; list.pop(0): O(n)
q.append(3); q.appendleft(0)
q.popleft(), q.pop()              # 0, 3
q.rotate(1)                       # [2,1]: right by 1
q[0], q[-1]                       # 2, 1
```

5 Python: strings and ordering

5.1 Strings

```
s = "  one  two\n"
s.strip(), s.split()              # 'one  two', ['one','two']
"a,,b,".split(",")                # ['a','','b','']
"-".join(["a", "b", "c"])         # 'a-b-c'
s.replace("one", "1")             # replace all occurrences
"banana".find("na")               # 2; -1 if absent
"banana".count("an")              # 2, non-overlapping
"abc".startswith("ab"), "abc".endswith("c")
"abc"[::-1], "AbC".lower()        # 'cba', 'abc'
ord("a"), chr(97)                 # 97, 'a'
```

Strings are immutable. Build many output fragments in a list, then join; repeated concatenation can be quadratic. `split()` collapses whitespace; `split(",")` keeps empty fields.

5.2 Formatting and conversions

```
x = 1 / 3
f"{x:.10f}"                       # '0.3333333333'
f"{7:03d}", f"{255:x}"            # '007', 'ff'
f"{13:08b}"                       # '00001101'
"  abc  ".strip()                 # 'abc'
int("-42"), float("1.25")         # -42, 1.25
str(123), list("abc")             # '123', ['a','b','c']
```

5.3 Sorting and key functions

```
a = [(2, 5), (1, 7), (2, 3)]
b = sorted(a)                     # lexicographic tuples
a.sort(key=lambda p: (p[0], -p[1]))
sorted(["bbb", "a", "cc"], key=len)
sorted([3, 1, 2], reverse=True)   # [3,2,1]
max(a, key=lambda p: p[1])        # item with largest second
min([], default=0)                # 0 instead of ValueError
```

Sorts are stable; equal keys keep their original order. `a.sort()` mutates and returns `None`; `sorted(a)` makes a new list.

5.4 Binary search in a sorted list

```
from bisect import bisect_left as bl, bisect_right as br
from bisect import insort
a, x = [1, 2, 2, 5], 2
bl(a, x), br(a, x)                # 1, 3
count = br(a, x) - bl(a, x)       # 2 occurrences
insort(a, 3)                      # [1,2,2,3,5]
```

Search: $O(\log n)$. Insertion: $O(n)$. This is not a balanced ordered set.

5.5 Extracting structured text

```
import re
re.findall(r"-?\d+", "x=-12, y=34")  # ['-12','34']
re.fullmatch(r"[A-Z]+\d+", "AB12") is not None
re.split(r"[,;]+", "a,b;;c")      # ['a','b','c']
```

6 Python: iteration and helpers

6.1 Combinatorial iterators

```
from itertools import (combinations, permutations, product,
    combinations_with_replacement, accumulate, chain, pairwise)
list(combinations("ABC", 2))      # AB, AC, BC (as tuples)
list(permutations("ABC", 2))      # AB, AC, BA, BC, CA, CB
list(product([0, 1], repeat=3))   # 000, 001, ..., 111
list(product("AB", [1, 2]))       # A1, A2, B1, B2
list(combinations_with_replacement("AB", 2))  # AA, AB, BB
list(accumulate([2, 3, 4], initial=0))  # [0,2,5,9]
list(chain.from_iterable([[1, 2], [3]]))  # [1,2,3]
list(pairwise([4, 7, 9]))         # [(4,7),(7,9)]
```

These return lazy iterators; `list(...)` above shows the results. Iterate directly to avoid storing all outputs. Repeated input values can give duplicate permutations.

6.2 Automatic memoization

```
from functools import cache

@cache
def f(i, mask):
    ...                          # translate the recurrence

f.cache_clear()                  # between different test cases
# Older Python: @lru_cache(None), from functools import
    ↪ lru_cache
```

Arguments must be hashable (e.g. ints, tuples, frozensets). Cache keys include arguments, not changing global variables. Clear between cases or make a new function. Recursion depth still applies; iterative traversal is safer for deep graphs.

6.3 Min-heaps and sorted merging

```
from heapq import (heapify, heappush, heappop, heappushpop,
                   heapreplace, nsmallest, nlargest, merge)
h = [5, 2, 8]
heapify(h)                       # O(n); minimum at h[0]
heappush(h, 3)
heappop(h)                       # 2; O(log n)
heappushpop(h, 4)                # push, then pop minimum
heapreplace(h, 6)                # pop minimum, then push
nlargest(2, [4, 1, 7]), nsmallest(2, [4, 1, 7])
list(merge([1, 4], [2, 3]))      # [1,2,3,4]; inputs sorted
```

Min-heap: use `(-priority, item)` for a max-heap. Tuples compare lexicographically; add a unique counter before non-comparable payloads. `heapreplace` needs a nonempty heap. Merging is lazy.

6.4 Comprehensions and generators

```
a = [3, 1, 3]
index = {x: i for i, x in enumerate(a)}  # last index of x
pairs = {(x, y) for x in range(2) for y in range(3)}
total = sum(x*x for x in a)      # generator: no extra list
first, *middle, last = [1, 2, 3, 4]
```

Use brackets for a list, braces for a set/dict, `(expr for ...)` for a generator.

7 Python: dates and reminders

7.1 Calendar arithmetic

```
from datetime import date, timedelta
a, b = date(2024, 2, 28), date(2024, 3, 1)
(b - a).days                     # 2 (leap year)
a + timedelta(days=2)            # 2024-03-01
a.weekday()                      # 2: Monday=0, Sunday=6
a.isoformat()                    # '2024-02-28'
date.fromisoformat("2024-03-01")  # date(2024,3,1)
from calendar import isleap, monthrange
isleap(2024), monthrange(2024, 2)[1]  # True, 29 days
```

`date` supports years 1–9999. `monthrange(y,m)` returns (weekday of day 1, days in month).

7.2 Topological order

```
from graphlib import TopologicalSorter, CycleError
pred = {"A": set(), "B": {"A"}, "C": {"A", "B"}}
order = list(TopologicalSorter(pred).static_order())
# ['A','B','C']; CycleError if the graph has a cycle
```

Input lists **predecessors**, not outgoing neighbours. With several valid orders, the returned order need not be lexicographically smallest.

7.3 JSON input

```
import json
obj = json.loads('{"a": [1, 2], "ok": true}')
obj["a"][0], obj["ok"]           # 1, True
json.dumps(obj)                  # encode back to JSON
```

7.4 C++ to Python: avoid surprises

- Indentation defines blocks; use 4 spaces.
- `and`/`or`/`not`, `True`/`False`/`None`; no `++`.
- `append`/`sort`/`reverse` mutate and return `None`.
- `==` compares values; `is` tests identity. Use `x is None` for a missing value.
- `(x,)` is a one-element tuple; `(x)` is just `x`.
- Iterating a dict gives keys. Do not change a dict/set's size while iterating it.
- Integers do not overflow; floats still round.

7.5 Complexities to remember

Operation	Cost
List index / last append or pop	$O(1)$ amort.
List search / front or middle insert	$O(n)$
Dict/set lookup, add, remove	$O(1)$ avg.
Deque append/pop at either end	$O(1)$
Heap push/pop	$O(\log n)$
Sort	$O(n \log n)$
List/string slice of length k	$O(k)$

Bounds above treat numbers/keys as fixed-size. Big integer arithmetic and hashing long keys depend on their size.

8 C++17 language reference

Examples assume `using namespace std;` and `using ll = long long;`. `<bits/stdc++.h>` is a GCC convenience header; it is not part of ISO C++. Prefer Python for exact huge integers.

8.1 Input, output and terminal debugging

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
int n; string word;
cin >> n >> word;                  // whitespace-separated
vector<int> a(n);
for (int& x : a) cin >> x;
cout << n << ' ' << word << '\n';
cout << fixed << setprecision(10) << 1.0 / 3 << '\n';
int x;
while (cin >> x) { /* integers until EOF */ }
```

Use `'\n'` for ordinary output; `endl` also flushes. Interactive output must flush: `cout << "? " << x << endl;`. After disabling synchronization, avoid mixing `cin`/`cout` with `scanf`/`printf`.

```
int n; cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string line;
getline(cin, line);                // next line, spaces retained
// getline(cin >> ws, line);        // skips ALL leading
    ↪ whitespace
cerr << "n = " << n << '\n';       // stderr: stays in terminal
dbg(n);                           // template: only with -DLOCAL
```

`>>` leaves the newline behind. `getline` removes its delimiter but preserves other spaces; `>> ws` also skips blank lines. Redirecting stdout with `> out.txt` leaves stderr visible. Raw `cerr` always runs; `dbg` is conditional on `LOCAL`.

8.2 Declarations, loops and references

```
using ll = long long;
using pii = pair<int, int>;
vector<int> a = {3, 1, 4};
const int MOD = 1000000007;
auto n = a.size();                 // unsigned size type
for (int i = 0; i < int(a.size()); ++i) { /* a[i] */ }
for (int x : a) { /* copy of each value */ }
for (int& x : a) ++x;              // change original elements
for (const auto& x : a) { /* read without copying */ }
auto copy = a;                     // independent vector copy
auto& alias = a;                   // another name for a
swap(a[0], a[1]);
```

`const T&` passes a read-only reference; `T&` allows mutation. Passing `vector<T>` by value copies it. Local scalar variables are uninitialized until assigned. Indices run from 0 to size-1; `v[i]` does not check bounds, `v.at(i)` does.

```
ll square(ll x) { return x * x; }  // define outside main
void add_one(vector<int>& a) {
    for (int& x : a) ++x;
}
int offset = 5;
auto shifted = [offset](int x) { return x + offset; };
auto change = [&](int x) { offset += x; };  // capture by ref
auto twice = [](auto x) { return x + x; }; // generic lambda
```

`[x]` copies the current value of `x`; `[&x]` refers to the original. A reference capture must not outlive its variable. Use `struct State { int x, y; };` for named fields.

8.3 Vectors, grids, pairs and tuples

```
vector<int> v(5, -1);              // five copies of -1
v.push_back(7); v.pop_back();
v.resize(8, 0);                    // change size; fill new items
v.reserve(100);                    // capacity only; no new items
v.assign(3, 9);                    // replace with [9,9,9]
v.insert(v.begin() + 1, 4);        // O(n), insert before index 1
v.erase(v.begin() + 1);            // O(n), remove index 1
int first = v.front(), last = v.back(); // require nonempty
vector<vector<int>> grid(3, vector<int>(4, 0));
pair<int, int> p = {2, 7};
auto [x, y] = p;                   // C++17 structured binding
tuple<int, int, string> t = {1, 2, "a"};
auto [u, w, name] = t;
get<0>(t) = 9;                     // index is compile-time
tie(x, y) = p;                     // assign existing variables
```

Each grid row is independent. `auto [x,y]` copies fields; `auto& [x,y]` refers to them. Pairs, tuples, vectors and strings compare lexicographically. Reallocating a vector invalidates its references and iterators; erasing invalidates positions at or after the erased element.

8.4 Strings and characters

```
string s = "banana";
s += '!'; s.pop_back();
s.substr(1, 3);                    // "ana": start, length
auto pos = s.find("na");           // 2; string::npos if absent
if (pos != string::npos) s.replace(pos, 2, "X");
s.insert(0, "pre"); s.erase(0, 3);
reverse(s.begin(), s.end());
string digits = to_string(123);
ll value = stoll("-123");          // decimal by default
int d = '7' - '0';                 // digit character -> 7
char c = 'A';
c = char(tolower((unsigned char)c));
bool digit = isdigit((unsigned char)c);
```

C++ strings are mutable byte sequences; single quotes create a character, double quotes a string. `substr` takes a length, not an end index. Check `npos` before converting a found position to `int`. `stoi`/`stoll` throw on invalid or out-of-range input.

8.5 Ordered sets, maps and multisets

```
set<int> s = {2, 5, 9};            // sorted, unique
s.insert(4); s.erase(9);
bool found = s.find(5) != s.end(); // C++17: no contains()
auto it = s.lower_bound(3);        // first >= 3, or end()
if (it != s.end()) cout << *it << '\n';
if (!s.empty()) cout << *s.begin() << ' ' << *s.rbegin();
multiset<int> ms = {2, 2, 7};
auto one = ms.find(2);
if (one != ms.end()) ms.erase(one); // erase ONE occurrence
ms.erase(2);                       // erase ALL remaining 2s
map<string, int> counts;
counts["apple"]++;                 // missing key inserts value 0
auto hit = counts.find("pear");    // lookup without insertion
if (hit != counts.end()) cout << hit->second;
for (const auto& [key, val] : counts) { /* sorted keys */ }
map<pair<int, int>, int> at;       // pair keys work directly
```

Ordered lookup/insert costs $O(\log n)$. Iterators are not indices: `it - s.begin()` is invalid, and `distance` is $O(n)$. Use `s.lower_bound(x)`, not the generic algorithm on set iterators. Never dereference `end()` or erase a failed `find`.

```
unordered_map<string, int> freq; // average O(1) lookup
freq.reserve(10000);
freq["a"]++;
bool has = freq.find("a") != freq.end();
unordered_set<int> seen;
seen.insert(4); seen.erase(4);
```

Hash containers have no sorted order; worst-case operations can be $O(n)$. Rehashing invalidates iterators. `pair` and `vector` have no standard C++17 hash; use an ordered map or supply a custom hash.

8.6 Queues, stacks, deques and heaps

```
deque<int> d = {1, 2};
d.push_front(0); d.push_back(3);
d.pop_front(); d.pop_back();       // O(1) ends; d[i] is O(1)
queue<int> q;
q.push(4); int a = q.front(); q.pop();
stack<int> st;
st.push(5); int b = st.top(); st.pop();
priority_queue<int> maxq;
priority_queue<int, vector<int>, greater<int>> minq;
maxq.push(7); maxq.push(2);
int largest = maxq.top(); maxq.pop();
using Item = pair<ll, int>;        // distance, vertex
priority_queue<Item, vector<Item>, greater<Item>> heap;
heap.push({12, 3});
auto [dist, vertex] = heap.top(); heap.pop();
```

All `pop()` operations return nothing: read the value first. Check `empty()` before `front`/`top`/`pop`. Heap push/pop costs $O(\log n)$; top is $O(1)$. Pair heaps break ties by the second field.

8.7 Sorting and binary search

```
vector<int> v = {5, 1, 4, 1};
sort(v.begin(), v.end());          // [1,1,4,5]
sort(v.begin(), v.end(), greater<int>()); // descending
stable_sort(v.begin(), v.end());  // equal keys keep input order
vector<pair<int, int>> a = {{1, 9}, {2, 3}, {3, 3}};
sort(a.begin(), a.end(), [](const auto& x, const auto& y) {
    if (x.second != y.second) return x.second < y.second;
    return x.first > y.first;    // tie: first field descending
});
```

A comparator must give a strict weak ordering: `cmp(x,x)` must be false; use <, not <=. Do not subtract keys to compare them: subtraction can overflow.

```
vector<int> a = {1, 2, 2, 5};      // sorted ascending
int x = 2;
auto lo = lower_bound(a.begin(), a.end(), x); // first >= x
auto hi = upper_bound(a.begin(), a.end(), x); // first > x
int first = int(lo - a.begin());  // 1; size() if none
int count = int(hi - lo);         // 2 equal values
bool exists = binary_search(a.begin(), a.end(), x);
int k = 2;                        // zero-based, 0 <= k < size
nth_element(a.begin(), a.begin() + k, a.end());
int kth = a[k];                   // kth smallest; not fully
    ↪ sorted
```

Binary search needs a range sorted using the same comparison. Iterator ranges are half-open: `[begin,end)`. Vector binary search costs $O(\log n)$; insertion or erasure in its middle costs $O(n)$.

8.8 Algorithm utilities

```
vector<int> v = {3, 1, 3, 4};
ll sum = accumulate(v.begin(), v.end(), 0LL);
int mx = *max_element(v.begin(), v.end()); // nonempty
int threes = count(v.begin(), v.end(), 3);
bool has = find(v.begin(), v.end(), 1) != v.end();
fill(v.begin(), v.end(), 7);
iota(v.begin(), v.end(), 0);      // 0,1,2,3
reverse(v.begin(), v.end());
v.erase(remove(v.begin(), v.end(), 2), v.end()); // drop all 2s
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());   // deduplicate
ll g = gcd(12LL, 18LL), l = lcm(4LL, 6LL);
vector<int> p = {0, 1, 2};        // start sorted
do { /* use p */ } while (next_permutation(p.begin(), p.end()));
```

`remove` and `unique` return a new logical end; `erase` actually shrinks the vector. `unique` only merges adjacent equal values. The initial value of `accumulate` selects its arithmetic type: `0LL` for `long long`.

8.9 Integer arithmetic, limits and modulo

```
int x = 1000000000, y = 1000000000;
ll product = 1LL * x * y;         // widen BEFORE multiplication
ll big = numeric_limits<ll>::max();
const ll INF = (1LL << 62);       // chosen sentinel, not
    ↪ infinity
int a = -7, b = 3;
int q = a / b, rem = a % b;       // -2, -1: division truncates
// For integer a, positive b:
int floor_div = a / b - (a % b < 0);
int ceil_div = a / b + (a % b > 0);
ll m = 1000000007, value = -12;   // m > 0
ll r = value % m; if (r < 0) r += m; // normalize to [0,m)
__int128 wide = (__int128)product * product; // GCC extension
ll reduced = (ll)(wide % m);
```

Signed overflow is undefined behavior, including intermediate expressions. Assigning an overflowing `int` product to `ll` does not repair it. Integer division truncates even when assigned to `double`: use `double(a)/b`. `pow` uses floating point; it is not exact integer exponentiation or modular exponentiation. C++17 has no standard arbitrary-precision integer type.

8.10 Bitmasks and bit counts

```
unsigned long long mask = 0;
int i = 5;                        // require 0 <= i < 64
mask |= 1ULL << i;                // set
mask &= ~(1ULL << i);             // clear
mask ^= 1ULL << i;                // toggle
bool set = (mask & (1ULL << i)) != 0;
int ones = __builtin_popcountll(mask); // GCC, 64-bit operand
if (mask) {
    int low = __builtin_ctzll(mask);  // lowest set bit index
    int high = 63 - __builtin_clzll(mask); // highest set bit
}
bitset<128> bits;
bits.set(7); bits.flip(2); bits.reset(7);
auto count = bits.count();
```

`ctz/clz` require a nonzero operand. Use unsigned operands for bit operations; shifting by a negative amount or by the type's width is invalid. The builtins above are GCC extensions; C++20 `popcount/countr_zero` are outside the C++17 baseline.

9 Mathematics: everyday formulas

9.1 Sums, ranges and averages

$n \geq 0$ is an integer; a is the first term, d the step, r the ratio. Empty sums are 0.

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2},$$
$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$
$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2,$$
$$\sum_{i=0}^{n-1} (a+id) = na + \frac{dn(n-1)}{2}.$$
$$\sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r-1} \quad (r \neq 1); \qquad r = 1 : \ n.$$

For $|r| < 1$, $\sum_{i=0}^{\infty} r^i = 1/(1-r)$. Modulo an integer, division requires an invertible denominator or an exact-division method; do not divide reduced residues normally.

For integers $L \leq R$ and $k > 0$, multiples of k in $[L, R]$:

$$\lfloor R/k \rfloor - \lfloor (L-1)/k \rfloor .$$

This also works for negative endpoints; use floor division. For real values $a_1, \ldots, a_n$, $n \geq 1$, minimizing over real x: $\sum_i |a_i - x|$ is minimized by a median; $\sum_i (a_i - x)^2$ by the mean $x = \sum_i a_i/n$.

9.2 Counting choices

Objects are distinct unless stated. n, k, s, u are nonnegative integers. $0! = 1$; $\binom{n}{k} = n!/[k!(n-k)!]$ for $0 \leq k \leq n$, and 0 outside that range or when the upper argument is negative.

What is counted	Count
Orders of n objects	$n!$
Choose k, order matters ($k \leq n$)	$n!/(n-k)!$
Choose k, order ignored	$\binom{n}{k}$
Subsets of n objects	2^n
Length-k words over n symbols	n^k
Paths: a right, b up	$\binom{a+b}{a}$
Choose k of $n \geq 1$ types, repeats	$\binom{n+k-1}{k}$
Circular orders, rotations identified ($n \geq 1$)	$(n-1)!$

The empty word has one realization, including for an empty alphabet: use $0^0 = 1$ in these counting formulas. Circular orders here distinguish reflections; necklaces with repeated colors need Burnside's lemma.

Repeated labels. If type i occurs $c_i \geq 0$ times and $\sum_i c_i = n$, distinct orders number $n!/\prod_i c_i!$.

Stars and bars. For $k \geq 1$ labeled boxes and total s:

$$\#\{x_1 + \cdots + x_k = s, \ x_i \geq 0\} = \binom{s+k-1}{k-1},$$
$$\#\{x_1 + \cdots + x_k = s, \ x_i \geq 1\} = \binom{s-1}{k-1} \quad (s \geq k).$$

The second count is 0 for $s < k$. With $0 \leq x_i \leq u$,

$$\sum_{j=0}^{k} (-1)^j \binom{k}{j} \binom{s - j(u+1) + k - 1}{k-1}.$$

9.3 Catalan, derangements and coverage

Catalan C_n. Balanced parentheses with n pairs, full ordered binary trees with n internal nodes, or triangulations of a convex $(n+2)$-gon ($n \geq 1$):

$$C_n = \frac{1}{n+1}\binom{2n}{n} \quad (n \geq 0).$$

$C_0, C_1, \ldots = 1, 1, 2, 5, 14, 42, 132, 429, \ldots$.

Derangements D_n. Permutations of n labels with no label in its original position:

$$D_0 = 1, \quad D_1 = 0, \quad D_n = (n-1)(D_{n-1} + D_{n-2}) \quad (n \geq 2).$$

$D_0, D_1, \ldots = 1, 0, 1, 2, 9, 44, 265, \ldots$.

Coupon collector. Independent uniform draws from $n \geq 1$ types, with replacement: expected draws to see all types is nH_n, where $H_n = \sum_{i=1}^{n} 1/i$.

9.4 Vectors, angles and distances

Points/vectors are in the plane. Write $a = (a_x, a_y)$, $b = (b_x, b_y)$; $|a|$ is Euclidean length.

$$a \cdot b = a_x b_x + a_y b_y, \qquad a \times b = a_x b_y - a_y b_x.$$

$a \times b > 0$: b is counterclockwise from a through an angle less than π; zero means parallel. Orientation of three points p, q, r is the sign of $(q-p) \times (r-p)$. $a \cdot b = 0$ means perpendicular for nonzero vectors.

$$|p-q|^2 = (p_x - q_x)^2 + (p_y - q_y)^2.$$

Compare squared distances when possible. Integer coordinates allow exact predicates; bound intermediate products and accumulated sums. Python integers are exact; C++ may need a wider intermediate type.

For nonzero vectors, the unsigned angle between them is $\operatorname{atan2}(|a \times b|, a \cdot b) \in [0, \pi]$. Direction of $a \neq 0$ is $\operatorname{atan2}(a_y, a_x)$. Convert degrees to radians by multiplying by $\pi/180$. Counterclockwise rotation through θ radians:

$$(x, y) \mapsto (x\cos\theta - y\sin\theta, x\sin\theta + y\cos\theta).$$

Point p to line/segment ab. Require $a \neq b$; put $v = b - a$, $t = (p-a) \cdot v/(v \cdot v)$. For the line, projection is $q = a + tv$. For the segment, first clamp t to $[0, 1]$. Distance is $|p - q|$. If $a = b$, use $|p - a|$.

9.5 Polygon area and lattice points

$p_0, \ldots, p_{n-1}$ are vertices in boundary order, $p_n = p_0$; A denotes nonnegative area.

$$A_2 = \sum_{i=0}^{n-1} p_i \times p_{i+1}, \qquad A = |A_2|/2.$$

For a simple polygon, $A_2 > 0$ means counterclockwise orientation. Triangle area is $|(b-a) \times (c-a)|/2$. A simple n-vertex polygon has interior-angle sum $(n-2)\pi$.

Pick's theorem. For a simple polygon with integer vertices and no holes, let I be its strictly interior lattice-point count and B its boundary lattice-point count:

$$A = I + B/2 - 1, \qquad B = \sum_{\text{edges}} \gcd(|\Delta x|, |\Delta y|).$$

Thus $I = (|A_2| - B + 2)/2$. The differences are between the endpoints of each edge; do not count shared vertices separately.

9.6 Triangles, circles and spheres

A nondegenerate triangle has side lengths $a, b, c > 0$, opposite angles α, β, γ, semiperimeter $s = (a+b+c)/2$ and area $A > 0$.

$$c^2 = a^2 + b^2 - 2ab\cos\gamma,$$
$$A = \sqrt{s(s-a)(s-b)(s-c)},$$
$$R = abc/(4A), \qquad r = A/s,$$
$$a/\sin\alpha = b/\sin\beta = c/\sin\gamma = 2R.$$

R is circumradius; r is inradius. With $a \leq b \leq c$, existence requires $a + b > c$; the largest angle is right, acute or obtuse according as $a^2 + b^2 - c^2$ is zero, positive or negative.

For a circle of radius $r \geq 0$ and central angle $0 \leq \theta \leq 2\pi$ in radians: circumference $2\pi r$, area πr^2, arc length $r\theta$, sector area $r^2\theta/2$, chord length $2r\sin(\theta/2)$. A sphere of radius r has surface area $4\pi r^2$ and volume $4\pi r^3/3$.

For floating-point geometry, choose tolerances for the input scale; one fixed epsilon is not universal. Clamp a cosine to $[-1, 1]$ before `acos`, and a tiny negative roundoff to 0 before `sqrt`. Never use an epsilon comparator for sorting unless it still defines a strict weak ordering.

Integer equations

N1 Solve $ax + by = c$ in integers

Symbols: a, b, c given integers, $(a, b) \neq (0, 0)$; $g = \gcd(a, b) > 0$ must divide c; (x_0, y_0) one solution; t any integer. All solutions:

$$x = x_0 + (b/g)t, \qquad y = y_0 - (a/g)t.$$

For bounds $L \leq x_0 + ht \leq R$, $h > 0$: $\lceil (L - x_0)/h \rceil \leq t \leq \lfloor (R - x_0)/h \rfloor$. Intersect t ranges for x, y.

Modulo and totients

Moduli are positive integers; exponents are integers $e \geq 0$.

N2 Count integers coprime to n: totient

Symbols: $n = \prod_i p_i^{e_i} \geq 1$; distinct primes p_i, exponents $e_i \geq 1$; d runs over positive divisors.

$\varphi(n) = \#\{1 \leq x \leq n : \gcd(x, n) = 1\}$; $\varphi(1) = 1$.

$$\varphi(n) = n \prod_{p|n} (1 - 1/p) = \prod_i p_i^{e_i - 1}(p_i - 1).$$

$\varphi(p^k) = p^{k-1}(p-1)$ for prime p, $k \geq 1$; $\sum_{d|n} \varphi(d) = n$.

$\gcd(a, b) = 1 \Rightarrow \varphi(ab) = \varphi(a)\varphi(b)$.

N3 Invert / reduce a power: Euler, Fermat

Symbols: a integer base, $m > 1$ modulus, p prime; a^{-1} is a value whose product with a is 1 modulo m.

Require $\gcd(a, m) = 1$**:**

$$a^{\varphi(m)} \equiv 1, \qquad a^e \equiv a^{e \bmod \varphi(m)} \pmod m.$$

Inverse: $a^{-1} \equiv a^{\varphi(m)-1} \pmod m$. Prime p: $a^p \equiv a \pmod p$ always; $a^{p-1} \equiv 1 \pmod p$ if $p \nmid a$.

N4 Solve / cancel congruences; merge by CRT

Symbols: a, b known integers; m, n moduli; x, y unknown integers; $g = \gcd(a, m)$; x_0 one solution.

$ax \equiv b \pmod m$: no solution if $g \nmid b$. Otherwise solve $(a/g)x \equiv b/g \pmod{m/g}$: $x = x_0 + t(m/g)$, $t = 0, \ldots, g-1$: g residues modulo m.

$$ax \equiv ay \pmod m \iff x \equiv y \pmod{m/g}.$$

CRT: $x \equiv a \pmod m$, $x \equiv b \pmod n$ are compatible iff $\gcd(m, n) \mid (b - a)$; period $\operatorname{lcm}(m, n)$.

N5 Power period / noncoprime bases

Symbols: $d = \operatorname{ord}_m(a)$: smallest positive exponent giving 1; q prime divisor of d; p^k prime-power modulus.

For $m > 1$, $\gcd(a, m) = 1$: $d \mid \varphi(m)$. Start $d = \varphi(m)$; repeatedly divide by q if $a^{d/q} \equiv 1 \pmod m$.

For $a \geq 1$, write $a = p^s u$, $p \nmid u$:

$$a^e \equiv \begin{cases} 0, & se \geq k, \\ p^{se} u^e, & se < k \end{cases} \pmod{p^k}.$$

s counts factors of p in a; reduce only u's exponent; merge prime powers by CRT.

Divisors and gcd sums

D1 Undo divisor sums: Möbius inversion

Symbols: $n \geq 1$ integer; d positive divisor; r distinct prime-factor count; f, F arithmetic functions.

$\mu(1) = 1$; $\mu(n) = 0$ if any prime square divides n; otherwise $\mu(n) = (-1)^r$. Also $\sum_{d|n} \mu(d) = [n = 1]$.

$$F(n) = \textstyle\sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) F(n/d).$$

Multiplicative f: $f(1) = 1$ and $f(ab) = f(a)f(b)$ for coprime a, b; compute from the prime powers in n.

D2 Count coprime values / pairs; sum gcds

Symbols: $N, M, m \geq 1$ integers; (x, y) ordered pair with $1 \leq x \leq N$, $1 \leq y \leq M$; $h_d = \lfloor N/d \rfloor \lfloor M/d \rfloor$.

$$\#\{1 \leq x \leq N : \gcd(x, m) = 1\} = \textstyle\sum_{d|m} \mu(d) \lfloor N/d \rfloor.$$
$$\#\{\gcd(x, y) = 1\} = \textstyle\sum_{d \leq \min(N,M)} \mu(d) h_d,$$
$$\textstyle\sum_{x=1}^{N} \sum_{y=1}^{M} \gcd(x, y) = \sum_{d \leq \min(N,M)} \varphi(d) h_d.$$

For gcd exactly $g \geq 1$, use the coprime-pair formula with $N \leftarrow \lfloor N/g \rfloor$, $M \leftarrow \lfloor M/g \rfloor$.

D3 Group floors / count $xy \leq N$

Symbols: $N \geq 1$ bound; l, r first/last index of a block; q common quotient; $s = \lfloor \sqrt{N} \rfloor$; $\tau(k)$ divisor count; x, y positive integers, ordered.

$$q = \lfloor N/l \rfloor, \qquad r = \lfloor N/q \rfloor.$$

Then $\lfloor N/i \rfloor = q$ for $l \leq i \leq r$. Start $l = 1$; advance $l = r + 1$: $O(\sqrt{N})$ blocks.

$$\textstyle\sum_{i=1}^{N} \lfloor N/i \rfloor = \sum_{k=1}^{N} \tau(k) = 2 \sum_{i=1}^{s} \lfloor N/i \rfloor - s^2.$$

Graph counts

Symbols: Simple undirected graph: no loops/parallel edges. V, E: vertex/edge sets; $n = |V|$; $m = |E|$; c: components; d_v: degree. $\#C_k$: simple k-cycles, rotations/reversals identified.

G1 Independent cycles / degrees / cuts

$$r = m - n + c, \qquad \sum_v d_v = 2m.$$

Symbols: r: independent cycles, or edges to remove for a spanning forest.

Even-degree edge subsets: 2^r, including empty. r is not the total simple-cycle count. Odd-degree vertex count is even. A cut separates a vertex subset from its complement; cycles cross it evenly. Cut-space dimension: $n - c$. Bridges belong to no cycle.

G2 Triangles

Symbols: $N(v)$: neighbours; $a_{uv} = |N(u) \cap N(v)|$: common-neighbour count. A: 0/1 adjacency matrix, diagonal 0. tr: diagonal sum.

$$\#C_3 = \frac{1}{3} \sum_{\{u,v\} \in E} a_{uv} = \frac{\operatorname{tr}(A^3)}{6}.$$

G3 Four-cycles / walks

$$\#C_4 = \tfrac{1}{2} \textstyle\sum_{u<v} \binom{a_{uv}}{2}, \\ = \frac{\operatorname{tr}(A^4) + 2m - 2\sum_v d_v^2}{8}.$$

$u < v$: unordered vertex pairs. Bipartite: sum pairs in one side only, omit $1/2$. Chords allowed. $(A^k)_{uv}$: length-k walks, allowing repetitions. $\operatorname{tr}(A^k)/(2k)$ is not a general simple-cycle formula.

G4 Complete-graph cycles

Symbols: K_n: every pair joined; $K_{a,b}$: complete bipartite, side sizes a, b. Ranges: $3 \leq k \leq n$ and $2 \leq k \leq \min(a, b)$, respectively.

$$\#C_k(K_n) = \tfrac{n!}{2k(n-k)!},$$
$$\#C_{2k}(K_{a,b}) = \tbinom{a}{k}\tbinom{b}{k}\tfrac{k!(k-1)!}{2}.$$

Hamilton cycles (all vertices): $(n-1)!/2$, $n \geq 3$. Complete directed graph with both arcs per pair: replace $2k$ by k for $k \geq 3$; reversal is distinct. Labeled simple graphs on n vertices: $2^{\binom{n}{2}}$.

Trees and planar graphs

G5 Labeled trees / prescribed degrees

Symbols: Labels distinguish vertices; $n \geq 2$. $d_i \geq 1$: required degree of label i; $\sum_i d_i = 2n - 2$.

Cayley: n^{n-2} labeled trees; n^{n-1} with a chosen root. Both counts are 1 for $n = 1$.

$$\#\text{trees with degrees } d_i = \frac{(n-2)!}{\prod_i (d_i - 1)!}.$$

Prüfer sequence: $n - 2$ labels; label i occurs $d_i - 1$ times. Leaves are exactly absent labels. For $n \geq 2$:

$$\#\text{leaves} = 2 + \sum_{v : d_v \geq 3} (d_v - 2).$$

G6 Spanning trees (Matrix–Tree)

Symbols: $\tau(G)$: spanning-tree count; C_n: one n-cycle. Laplacian L: $L_{ii} = d_i$; $L_{ij} = -1$ if adjacent, 0 otherwise. $L^{(j)}$: remove row and column j; $\det$: determinant.

$$\begin{aligned} \tau(G) &= \det L^{(j)}, \\ \tau(K_n) &= n^{n-2} && (n \geq 2), \\ \tau(K_{a,b}) &= a^{b-1} b^{a-1} && (a, b \geq 1), \\ \tau(C_n) &= n && (n \geq 3). \end{aligned}$$

Disconnected G: 0. Full $\det L = 0$. Elimination modulo a prime: $O(n^3)$; row swaps flip the sign. Counts all spanning trees, not only minimum-weight trees.

G7 Planar faces / edge bounds

Symbols: f: faces including outside; planar means drawable without crossings.

$$n - m + f = 1 + c.$$

For simple planar graphs, $n \geq 3$:

$$m \leq 3n - 6; \qquad m \leq 2n - 4 \text{ if bipartite.}$$

Bounds do not prove planarity. Triangulation (all faces triangles): $m = 3n - 6$, $f = 2n - 4$. Every nonempty simple planar graph has a vertex of degree at most 5.

Partitions and symmetry

C1 Partitions / onto functions

Symbols: n: distinct objects; k: groups/cycles. $c(n,k)$: permutations with k cycles (unsigned Stirling I). $S(n,k)$: k nonempty unlabeled groups (Stirling II). Bell $B_n = \sum_k S(n,k)$: all partitions.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),$$
$$S(n,k) = S(n-1,k-1) + kS(n-1,k).$$

Both $(0,0) = 1$; other $k = 0$ or $k > n$ entries: 0. Use $0^0 = 1$.

$$k!S(n,k) = \sum_{j=0}^{k} (-1)^j \binom{k}{j} (k-j)^n.$$

Onto: n objects map to k labeled targets, all used; j: excluded targets in inclusion–exclusion.

C2 Prescribed permutation cycles

Symbols: $a_j \geq 0$: cycles of length j; $\sum_j j a_j = n$. $H_n = \sum_{j=1}^{n} 1/j$.

$$\#\text{permutations} = \frac{n!}{\prod_j j^{a_j} a_j!}.$$

Uniform random permutation: expected j-cycles $= 1/j$ for $1 \leq j \leq n$; expected total cycles $= H_n$.

C3 Symmetric objects (Burnside)

Symbols: X: objects; G: finite group of allowed symmetries. $\mathrm{Fix}(g)$: objects unchanged by symmetry g. Orbit: objects equivalent under the symmetries.

$$\#\text{orbits} = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|.$$

Do not just divide $|X|$ by $|G|$. A position permutation with z cycles fixes q^z colorings with q colors.

C4 Necklaces / bracelets

Symbols: $n \geq 1$: positions; q: colors; r: rotation offset; d: divisor of n. R, F: total fixed colorings over rotations, reflections; φ: totient (N2).

$$R = \sum_{r=0}^{n-1} q^{\gcd(n,r)} = \sum_{d \mid n} \varphi(d) q^{n/d}.$$

Necklaces: R/n (rotations equivalent). Bracelets: $(R+F)/(2n)$ (also reflections).

$$F = \begin{cases} nq^{(n+1)/2}, & n \text{ odd}, \\ \frac{n}{2}(q^{n/2} + q^{n/2+1}), & n \text{ even}. \end{cases}$$

Modulo prime p: invert denominator $d = n$ or $2n$ only if $p \nmid d$; otherwise compute numerator modulo pd and integer-divide the residue by d (B5).

Sums and sequences

S1 Weighted geometric sums

Symbols: $n \geq 0$: integer upper limit; i: index; r: ratio, $r \neq 1$.

$$\sum_{i=1}^{n} i r^i = \frac{r(1-(n+1)r^n + nr^{n+1})}{(1-r)^2}.$$

At $r = 1$: $n(n+1)/2$. For $|r| < 1$: $\sum_{i \geq 1} i r^i = r/(1-r)^2$. Modulo $m > 0$: denominator must be invertible.

S2 Pair distances and products

Symbols: $a_1, \ldots, a_n$: real values; $i < j$: each pair once. First formula: sort $a_1 \leq \cdots \leq a_n$.

$$\sum_{i<j} |a_i - a_j| = \sum_{i=1}^{n} (2i - n - 1) a_i.$$

Without sorting:

$$\sum_{i<j} (a_i - a_j)^2 = n \sum_i a_i^2 - \left(\sum_i a_i \right)^2,$$
$$\sum_{i<j} a_i a_j = \frac{(\sum_i a_i)^2 - \sum_i a_i^2}{2}.$$

Manhattan distances: apply the first formula per coordinate.

S3 Fibonacci doubling and sums

Symbols: $F_0 = 0$, $F_1 = 1$, $F_{n+2} = F_{n+1} + F_n$; $n, k \geq 0$: integer indices.

$$F_{2k} = F_k(2F_{k+1} - F_k),$$
$$F_{2k+1} = F_k^2 + F_{k+1}^2.$$

$\sum_{i=0}^{n} F_i = F_{n+2} - 1$; $\sum_{i=0}^{n} F_i^2 = F_n F_{n+1}$. $\gcd(F_a, F_b) = F_{\gcd(a,b)}$ for integer $a, b \geq 1$. Doubling takes $O(\log n)$ arithmetic steps. Modulo m: all identities work except the gcd identity.

S4 Convolution and recurrences

Symbols: $A(x) = \sum_{n \geq 0} a_n x^n$, $B(x) = \sum_{n \geq 0} b_n x^n$; $[x^n]$: coefficient of x^n; x: formal variable.

$$[x^n](AB) = \sum_{i=0}^{n} a_i b_{n-i}.$$

For constants c, d and $a_n = ca_{n-1} + da_{n-2}$, $n \geq 2$:

$$A(x) = \frac{a_0 + (a_1 - ca_0)x}{1 - cx - dx^2}.$$

Formal series: no convergence requirement.

Binomials and modular counting

Symbols: $\binom{n}{r}$: choose r of n objects; parameters and indices are integers. Upper arguments are nonnegative; $\binom{n}{r} = 0$ outside $0 \leq r \leq n$.

B1 Binomial sums

For nonnegative integer parameters:

$$\sum_{i=0}^{n} \binom{i}{k} = \binom{n+1}{k+1},$$
$$\sum_k \binom{a}{k}\binom{b}{r-k} = \binom{a+b}{r},$$
$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n},$$
$$\sum_{k=0}^{n} k\binom{n}{k} = n\,2^{n-1} \quad (n \geq 1),$$
$$\sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0 \quad (n \geq 1).$$

B2 Prime factors and trailing zeros

Symbols: p: prime; $v_p(N)$: exponent of p in integer $N > 0$; $0 \leq r \leq n$ (Legendre).

$$v_p(n!) = \sum_{j \geq 1} \lfloor n/p^j \rfloor,$$
$$v_p\tbinom{n}{r} = v_p(n!) - v_p(r!) - v_p((n-r)!).$$

Base $b = \prod_i p_i^{e_i} > 1$, with distinct primes p_i:

$$\text{Trailing zeros of } n! = \min_i \lfloor v_{p_i}(n!)/e_i \rfloor.$$

B3 Binomial mod a prime (Lucas)

Symbols: p: prime modulus; n_i, r_i: base-p digits, $n = \sum_i n_i p^i$, $r = \sum_i r_i p^i$, $0 \leq n_i, r_i < p$.

$$\binom{n}{r} \equiv \prod_i \binom{n_i}{r_i} \pmod{p}.$$

Any $r_i > n_i$ gives 0. Factorial inverses require indices $< p$; $p!$ has no inverse modulo p.

B4 Binomial mod a prime power

Symbols: p: prime; $k \geq 1$; $0 \leq r \leq n$; $e = v_p\binom{n}{r}$; $U(t) = t!/p^{v_p(t!)}$.

If $e \geq k$, answer 0. Otherwise:

$$\binom{n}{r} \equiv p^e U(n) U(r)^{-1} U(n-r)^{-1} \pmod{p^k}.$$

Strip p factors before reducing/inverting U modulo p^k. Combine prime powers by CRT.

B5 Exact modular division

Symbols: A: integer numerator; $d > 0$: divisor with $d \mid A$; $m > 0$: modulus; $A \bmod (dm) \in [0, dm)$.

$$(A/d) \bmod m = \frac{A \bmod (dm)}{d}.$$

Integer-divide the residue by d; no inverse needed.

Probability and expectation

Symbols: X, Y, X_i: random variables; P: probability; E: expectation; Var: variance; Cov: covariance. All required moments finite.

P1 Expectation and variance

Symbols: a, b: constants; $I \in \{0,1\}$: event indicator, $P(I=1)=p$.

$E[\sum_i X_i] = \sum_i E[X_i]$, without independence.

$$\mathrm{Var}(X) = E[X^2] - E[X]^2,$$
$$\mathrm{Cov}(X,Y) = E[XY] - E[X]E[Y].$$

$$\mathrm{Var}\left(\sum_i X_i\right) = \sum_i \mathrm{Var}(X_i) + 2\sum_{i<j} \mathrm{Cov}(X_i, X_j).$$

$\mathrm{Var}(aX+b) = a^2\,\mathrm{Var}(X)$. Independence gives covariance 0.

$E[I] = p$, $\mathrm{Var}(I) = p(1-p)$.

P2 Tail sums and extrema

For nonnegative integer-valued X:

$$E[X] = \textstyle\sum_{k\geq 1} P(X \geq k),$$
$$E[X^2] = \textstyle\sum_{k\geq 1} (2k-1) P(X \geq k).$$

For independent X_i, real threshold t:

$$P(\max_i X_i \leq t) = \textstyle\prod_i P(X_i \leq t),$$
$$P(\min_i X_i > t) = \textstyle\prod_i P(X_i > t).$$

P3 Independent trials: binomial

Symbols: n independent trials, success probability $p \in [0,1]$; X: success count; k: outcome, $0 \leq k \leq n$.

$$P(X=k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

$E[X] = np$; $\mathrm{Var}(X) = np(1-p)$.

P4 First success: geometric

Symbols: Independent trials, success probability $p \in (0,1]$; X: trials through first success; k: outcome, $k \geq 1$.

$$P(X=k) = (1-p)^{k-1} p.$$

$E[X] = 1/p$; $\mathrm{Var}(X) = (1-p)/p^2$.

P5 No replacement: hypergeometric

Symbols: $N > 1$ objects, $0 \leq K \leq N$ marked; $0 \leq n \leq N$ uniform draws without replacement; X: marked count; k: outcome; $p = K/N$.

$$P(X=k) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}.$$

$E[X] = np$; $\mathrm{Var}(X) = np(1-p)(N-n)/(N-1)$.

P6 Probability bounds

Symbols: $t > 0$: threshold; A_i: events.

Markov ($X \geq 0$): $P(X \geq t) \leq E[X]/t$. Chebyshev: $P(|X - E[X]| \geq t) \leq \mathrm{Var}(X)/t^2$. Union: $P(\bigcup_i A_i) \leq \sum_i P(A_i)$; no independence needed.

10 Algorithm roadmap

Use these notes to choose a technique and check its assumptions. n, m denote vertex/edge counts in graph bounds, or input lengths elsewhere. Running times count algorithmic operations; Python object costs, big integers and memory can change what is practical.

10.1 Search, arrays and greedy choices

Binary search on the answer. Find a monotone predicate first. For the first true integer in $[l, r)$, maintain that every value below l is false and the answer is at most r. Check the endpoint cases and return the sentinel r if no value in the interval works. Cost is $O(T \log(r-l+1))$ when one predicate evaluation costs T. For real answers, stop after a justified number of iterations or when the interval is below the required error; keep false/true endpoint roles.

Prefix and difference arrays. Define $P[0]=0$, $P[i+1] = P[i] + a[i]$: sum of $[l, r)$ is $P[r] - P[l]$. To add v to $[l, r)$ offline, do $D[l]{+}{=}v$, $D[r]{-}{=}v$, then prefix-sum D. Allocate the sentinel at n. For online updates and range queries, consider a Fenwick or segment tree.

Two pointers. Linear time requires that each pointer only moves forward and that validity changes monotonically. A sum-bounded window works for nonnegative values; negative values break that argument. A monotone deque gives all fixed-window minima in $O(n)$; remove expired indices, not just expired values.

Coordinate compression. Sort unique values and replace each by its rank: $O(n \log n)$. Order/equality are preserved; actual distances and gaps are not. Recover differences from the original coordinates.

Greedy. Establish an exchange argument or invariant before committing. Maximum number of compatible intervals: increasing end time. Maximum *weight* of compatible intervals needs DP. For a sweep, define tie order: touching half-open intervals do not overlap; touching closed intervals do. Try tiny counterexamples.

10.2 Dynamic programming

Write down **state meaning, transition, base cases, evaluation order**. Time is roughly reachable states times choices per state; also count the cost of copying strings, masks or containers in each transition. For minimum cost, impossible states need infinity, not 0. For maximum with negative values, use negative infinity. For counting, use 0 and initialize the empty construction explicitly.

Structure	Typical state / cost
One sequence, best prefix	$dp[i]$; depends on transition
Two sequences, LCS/edit distance	$dp[i][j]$; $O(nm)$
Each item once, capacity W	$dp[w]$; $O(nW)$
Subsets of n items	$dp[mask]$; exponential
Interval split / merge	$dp[l][r]$; often $O(n^3)$
Digits up to a bound	position, tight, started, property

Knapsack loop direction: descending capacities for 0/1 items; ascending for unlimited copies. Counting coin combinations and ordered sequences uses different outer loops. LIS uses lower-bound insertion for strictly increasing, upper-bound insertion for nondecreasing.

Memoization. Cache only complete states; include every changing constraint or clear the cache between cases. Recursive evaluation needs an acyclic dependency graph and manageable depth. Two-row DP saves memory when older rows are unused; reconstruction may need extra storage.

10.3 Graph selection and invariants

Edge weights / task	Method	Time
Every edge costs 1	BFS	$O(n+m)$
Weights only 0 or 1	0–1 BFS	$O(n+m)$
Nonnegative weights	Dijkstra	$O((n+m)\log n)$
Negative weights	Bellman–Ford	$O(nm)$
Acyclic directed graph	Topological DP	$O(n+m)$
All-pairs distances	Floyd–Warshall	$O(n^3)$
Connectivity under merges	DSU	amort. $O(\alpha(n))$
Minimum spanning tree	Kruskal	$O(m \log m)$

BFS: mark visited on enqueue so each vertex enters the queue once. Multiple sources start at distance 0. In a grid, include all state that affects future moves: position alone may omit direction, keys or time. For 0–1 BFS, put a relaxed 0-cost edge at the deque front and a 1-cost edge at its back.

Dijkstra: skip a popped heap entry when its distance differs from the current best distance. Relax edges only on a strict improvement. Negative weights invalidate the settled-distance argument. Keep parent links if the path itself is needed.

Negative cycles: Bellman–Ford's extra relaxation only proves a cycle relevant to a source when that edge is reachable from it. An answer is unbounded only if the cycle can also reach the target. Floyd–Warshall

uses the intermediate vertex as its outermost loop.

Dependencies and components: Kahn's topological sort removes zero-indegree vertices; fewer than n removals means a directed cycle. SCC contraction produces a DAG. DSU handles undirected connectivity under additions; it does not answer directed reachability or ordinary deletions. α in its bound is the inverse Ackermann function. Kruskal on a disconnected graph gives a forest, not a spanning tree.

Euler walk: degree conditions also need connectivity among vertices incident to edges (weak connectivity for a directed graph). Undirected: zero or two odd degrees. Directed: balanced indegree/outdegree, or one start with one extra outgoing edge and one end with one extra incoming. Track edge IDs when parallel edges exist.

Bipartite matching: augmenting paths grow a matching one edge at a time. In a bipartite graph, minimum vertex-cover size equals maximum matching size; maximum independent-set size is n minus that size. For shortest paths *to* a target, reverse directed edges and run from the target. Minimum spanning trees solve total connection cost, not routes.

10.4 Trees and repeated transitions

A connected undirected graph on n vertices with $n-1$ edges is a tree. Root it iteratively; record parents and an order with parents before children. Reverse that order for subtree sizes and bottom-up DP. Every pair has a unique path: accumulate weights along that path; no shortest-path heap is needed just to compute root distances.

Euler intervals. With entry time $tin[v]$ and exclusive exit $tout[v]$, the subtree is $[tin[v], tout[v])$. Define the traversal convention consistently; an entry/exit-event sequence has different indexing.

Binary lifting. Store the 2^j-th ancestor/successor for each vertex. Preprocessing $O(n \log K)$; a jump of up to K steps costs $O(\log K)$. LCA by lifting depths, then lifting both vertices together costs $O(\log n)$ per query. Weighted distance is $d[a] + d[b] - 2d[\mathrm{lca}(a, b)]$, where d is root distance.

Rerooting. Moving the root across an unweighted tree edge into a subtree of size s changes the sum of distances by $n - 2s$. That edge belongs to $s(n-s)$ unordered vertex-pair paths. Two farthest-vertex traversals find a diameter for unweighted trees, or weighted trees with nonnegative edge lengths; do not apply this shortcut to arbitrary graphs or negative edge lengths.

10.5 String techniques

Start with Python string operations or C++ `find` for ordinary parsing and a small number of searches; account for copied slices.

Frequencies/parity: two words are anagrams iff their frequency tables match. A multiset can form a palindrome iff at most one count is odd.

Rotations: equal-length a, b are rotations iff b occurs in $a + a$.

KMP/prefix function: $\pi[i]$ is the longest proper prefix that is also a suffix of $s[0..i]$. It gives linear pattern matching and periods. A candidate period $p = n - \pi[n-1]$ repeats the entire nonempty string iff $n \bmod p = 0$. **Rolling hash:** collisions are possible; use a second hash or verify candidate matches when exact correctness matters. **Trie:** prefix queries cost the word length, but a Python dictionary per node can be memory-heavy. Sorting words and binary searching a prefix range can be simpler. State the allowed alphabet before constructing a sentinel such as `prefix + "{"`.

10.6 Geometry decisions

Use orientation and squared distances for exact integer predicates. Handle collinear, coincident and endpoint cases deliberately. Choose whether boundary points count as inside before coding a polygon test. Sorting points by coordinates followed by a monotone-chain hull costs $O(n \log n)$; the cross-product comparison decides whether collinear hull boundary points are kept. Polygon vertices must be in boundary order for shoelace area. For farthest-pair questions, first restrict to hull vertices; a hull alone does not make an all-pairs scan linear. Avoid a floating-point tolerance inside an ordering comparator unless transitivity is preserved. Check every product and accumulated sum for C++ overflow; delay square roots and division where possible.

10.7 Finite impartial games

Under normal play, a state with no legal move loses. A state wins iff some move reaches a losing state. Evaluate states only after their successors; games with cycles may have draws and need more than this acyclic recurrence. For independent finite impartial games, define $g(s) = \mathrm{mex}\{g(t) : s \to t\}$, where mex is the smallest nonnegative integer absent from the set. The combined position loses iff the xor of its Grundy values is 0. For Nim, each heap's Grundy value is its size. Misère play and interacting subgames need separate reasoning.

10.8 Testing and the last minute

Compare a fast solution with a small, obviously correct brute force on many generated inputs. Save the random seed and failing input. Check both exit status and output; a crashing reference is not an agreement. Test empty/minimum cases when allowed, repeated values, all equal values, negative values, disconnected graphs, boundary coordinates and maximal depth. For floating output, compare with the specified absolute/relative tolerance.

Before submission: correct file/language, required test-case loop, index base, integer width, output spelling, and no debug text on stdout. For interactive problems, flush each query and use the supplied interactor; ordinary sample redirection does not simulate an interactive judge.